Problemas NP-Completos e Aproximações: Fundamentação, Algoritmos Exatos, Aproximados e Metaheurísticas em *Molecular Docking*

Igor, Maria Eduarda, Kairo Henrique, Heitor Henrique e Jade Souza
Departamento de Computação e Mecânica (DECOM-DV) – CEFET-MG, Divinópolis, MG
Estratégias Avançadas de Projeto e Análise de Algoritmos (Prof. Michel Pires da Silva)

***Resumo*—Este trabalho estuda os problemas NP-Completos e os métodos usados para lidar com o alto custo computacional de resolvê-los, tendo como base os Capítulos 34 e 35 do livro de Cormen et al. Na parte teórica, são apresentadas as classes P, NP, NP-Difícil e NP-Completo, a verificação em tempo polinomial por certificados, a discussão sobre P $\neq$ NP, as reduções polinomiais (CIRCUIT-SAT $\leq_P$ SAT $\leq_P$ 3-CNF-SAT $\leq_P$ CLIQUE $\leq_P$ VERTEX-COVER $\leq_P$ HAM-CYCLE $\leq_P$ TSP) e o cálculo da explosão combinatória. Na parte prática, são comparadas três estratégias implementadas em Python: algoritmos exatos (Força Bruta, *Branch-and-Bound* e Held-Karp), algoritmos de aproximação (`APPROX-VERTEX-COVER`, `APPROX-TSP-TOUR` e `GREEDY-SET-COVER`) e a metaheurística *Simulated Annealing* aplicada a um modelo simplificado de *Molecular Docking*.**

***Index Terms*—NP-Completude, Reduções Polinomiais, Explosão Combinatória, Algoritmos de Aproximação, *Simulated Annealing*.**

I. Introdução

Vários problemas clássicos de grafos possuem algoritmos exatos e rápidos. Para achar o menor caminho entre dois vértices com pesos não negativos, o algoritmo de Dijkstra resolve o problema em tempo $O(|E| + |V| \log |V|)$. Para conectar todos os vértices com o menor peso total de arestas, Prim e Kruskal encontram a Árvore Geradora Mínima em $O(|E| \log |V|)$ [1]. Nesses casos, o tempo de execução cresce de forma polinomial e permite processar grafos grandes em poucos segundos.

Por outro lado, existem problemas parecidos que não têm algoritmo eficiente conhecido. Se mudarmos o problema de caminhos mínimos para obrigar que a rota passe por todas as n cidades exatamente uma vez e volte ao início com o menor custo, temos o Problema do Caixeiro-Viajante (*Traveling Salesperson Problem* – TSP). Situação parecida acontece na Cobertura de Vértices (*Vertex Cover*), em que buscamos o menor conjunto de vértices que cubra todas as arestas do grafo, na Cobertura de Conjuntos (*Set Cover*) e no problema de encaixe molecular (*Molecular Docking*) [2], [3], onde se busca a posição e a rotação de uma molécula que minimizem a energia de ligação com uma proteína.

Na década de 1970, Cook [4], Karp [5] e Levin [6] provaram que esses problemas pertencem à classe NP-Completo (na forma de decisão) ou NP-Difícil (na forma de otimização). Todos eles são ligados por reduções em tempo polinomial: se alguém achar um algoritmo polinomial para um deles, todos os outros também terão solução polinomial.

Como não se conhece algoritmo polinomial para nenhum problema NP-Completo, achar a solução ótima exata exige testar um número de combinações que cresce de forma exponencial (2^n subconjuntos na Cobertura de Vértices) ou fatorial ($(n-1)!/2$ rotas no TSP simétrico). Com $n = 100$ cidades no TSP, existem cerca de $4{,}66 \times 10^{155}$ rotas, o que torna a busca exata inviável mesmo nos computadores mais rápidos.

Para contornar esse limite na prática, costumam ser usadas três abordagens:

1) *Algoritmos exatos com poda ou programação dinâmica:* métodos como *Branch-and-Bound* e Held-Karp [7] cortam buscas repetidas e acham o valor ótimo para entradas pequenas ($n \leq 20$), mas ainda sofrem com tempo exponencial e falta de memória RAM.
2) *Algoritmos de aproximação:* algoritmos polinomiais (Cap. 35 de Cormen et al. [1]) que entregam uma solução próxima da ótima com um limite matemático de erro $\rho(n)$.
3) *Metaheurísticas:* buscas probabilísticas como o *Simulated Annealing* [8], [9], usadas em problemas contínuos ou com muitos mínimos locais, onde não há garantia teórica de erro, mas busca-se uma boa solução dentro de um número fixo de avaliações.

O objetivo deste trabalho é estudar a base teórica da NP-Completude e comparar na prática o tempo de execução e a qualidade das soluções dessas três abordagens. Os objetivos específicos são: (i) apresentar as classes P, NP, NP-Difícil e NP-Completo, os verificadores polinomiais e a questão P $\neq$ NP; (ii) demonstrar as reduções da cadeia CIRCUIT-SAT $\leq_P$ SAT $\leq_P$ 3-CNF-SAT $\leq_P$ CLIQUE $\leq_P$ VERTEX-COVER $\leq_P$ HAM-CYCLE $\leq_P$ TSP e o crescimento de 2^n e $(n-1)!/2$; (iii) implementar soluções exatas e verificadores para TSP e Cobertura de Vértices; (iv) implementar os algoritmos de aproximação do Capítulo 35 e medir suas razões de aproximação; e (v) aplicar o *Simulated Annealing* ao estudo de caso inspirado em *Molecular Docking*.

O restante do texto está dividido da seguinte forma: a Seção II traz a fundamentação teórica; a Seção III apresenta os pseudocódigos e diagramas dos algoritmos; a Seção IV faz a análise assintótica de tempo e espaço e as provas de

aproximação; a Seção V mostra a implementação e os testes; a Seção VI resume as conclusões; e o Apêndice traz o guia rápido (*Cheat Sheet*).

II. Fundamentação Teórica

A. Problemas de Decisão e de Otimização

Na prática, problemas como o TSP ou a Cobertura de Vértices são de otimização, pois pedem o menor custo ou o menor conjunto possível. Porém, a teoria da NP-Completude do Capítulo 34 de Cormen et al. [1] trabalha com **problemas de decisão**, cuja resposta é apenas `SIM` (1) ou `NÃO` (0).

Todo problema de otimização pode ser transformado em um problema de decisão colocando um limite k sobre o valor buscado:

- `VERTEX-COVER`: dado um grafo não dirigido $G = (V, E)$ e um inteiro k, existe uma cobertura de vértices em G de tamanho no máximo k?
- `TSP`: dado um grafo completo $G = (V, E)$, os custos das arestas $c(i, j) \geq 0$ e um inteiro k, existe um ciclo hamiltoniano em G com custo total no máximo k?

Se fosse possível resolver a versão de otimização em tempo polinomial, bastaria achar o valor ótimo C^* e comparar se $C^* \leq k$ em $O(1)$ para responder ao problema de decisão. Logo, se o problema de decisão já for difícil, o de otimização é pelo menos tão difícil quanto ele. No livro, cada instância é codificada como uma cadeia binária $x \in \{0,1\}^*$, e o problema de decisão é visto como uma linguagem $L \subseteq \{0,1\}^*$ formada pelas entradas x que têm resposta `SIM` ($x \in L$).

B. Classes P e NP, Certificados e Verificadores

Definição 1 (Classe P [1])**.** A classe P é o conjunto das linguagens $L \subseteq \{0,1\}^*$ que podem ser decididas por um algoritmo determinístico em tempo polinomial $O(n^k)$, onde $n = |x|$ é o tamanho da entrada e k é uma constante.

Já a classe NP (*tempo polinomial não determinístico*) reúne os problemas cuja solução pode ser verificada em tempo polinomial.

Definição 2 (Verificador e Classe NP [1])**.** Um algoritmo de verificação $A(x, y)$ recebe a instância x e uma cadeia binária y chamada **certificado**. Uma linguagem L pertence a NP se existe um verificador de tempo polinomial A e uma constante c tais que:

$$L = \{x \in \{0,1\}^* : \exists y \text{ com } |y| = O(|x|^c) \text{ e } A(x, y) = 1\}. \tag{1}$$

Nos algoritmos implementados em `src/verification/verifier.py`, isso fica claro:

- Em `VERTEX-COVER`, o certificado é o subconjunto de vértices $V' \subseteq V$. O verificador confere se $|V'| \leq k$ e percorre as arestas $(u, v) \in E$ checando se $u \in V'$ ou $v \in V'$, gastando tempo $O(|V| + |E|)$.
- Em `TSP`, o certificado é a sequência de n vértices do ciclo. O verificador checa em $O(n)$ se todos os vértices aparecem uma única vez e se a soma dos pesos das arestas é menor ou igual a k.

Como qualquer problema em P já pode ser resolvido do zero em tempo polinomial sem precisar ler o certificado y, vale que P $\subseteq$ NP.

C. Reduções, NP-Difícil e NP-Completo

Definição 3 (Redução em Tempo Polinomial [1])**.** Uma linguagem L_1 é redutível em tempo polinomial a L_2 ($L_1 \leq_P L_2$) se existe uma função $f : \{0,1\}^* \to \{0,1\}^*$ calculável em tempo polinomial tal que, para todo $x \in \{0,1\}^*$:

$$x \in L_1 \iff f(x) \in L_2. \tag{2}$$

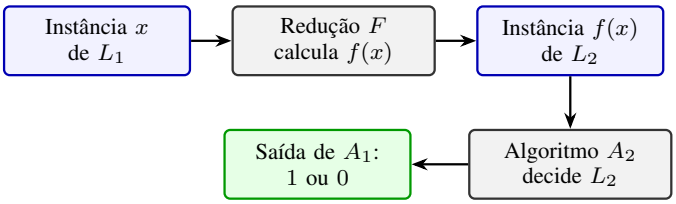


Figura 1. Ideia da redução polinomial $L_1 \leq_P L_2$ (Figura 34.5 de Cormen et al. [1]).

Lema 1 (Lema 34.3 de Cormen et al. [1])**.** *Se $L_1, L_2 \subseteq \{0,1\}^*$ são tais que $L_1 \leq_P L_2$, então $L_2 \in$ P $\implies$ $L_1 \in$ P.*

Demonstração. Sejam A_2 um algoritmo que decide L_2 em $O(|y|^k)$ e F o algoritmo de redução que gera $y = f(x)$ em $O(|x|^c)$ (Figura 1). Como F dá no máximo $O(|x|^c)$ passos, o tamanho de y é $|y| = O(|x|^c)$. Assim, $A_2(f(x))$ roda em tempo $O((|x|^c)^k) = O(|x|^{ck})$, e o algoritmo composto A_1 decide L_1 em tempo polinomial. □

Definição 4 (NP-Completo e NP-Difícil [1])**.** Uma linguagem $L \subseteq \{0,1\}^*$ é **NP-Completa** (NPC) se: (1) $L \in$ NP; e (2) $L' \leq_P L$ para todo $L' \in$ NP. Se L cumpre a condição 2 mas não precisa estar em NP, dizemos que L é **NP-Difícil**.

Teorema 1 (Teorema 34.4 de Cormen et al. [1])**.** *Se algum problema NP-Completo é resolvível em tempo polinomial, então P = NP. De modo equivalente, se algum problema em NP não está em P, então nenhum problema NP-Completo pode ser resolvido em tempo polinomial.*

Demonstração. Suponha que exista $L \in$ NPC tal que $L \in$ P. Para qualquer $L' \in$ NP, temos $L' \leq_P L$ pela propriedade 2 de NPC. Pelo Lema 1, $L \in$ P implica $L' \in$ P. Logo, NP $\subseteq$ P, e como P $\subseteq$ NP, segue que P = NP. A segunda parte do enunciado é a contrapositiva da primeira. □

D. A Questão P ≠ NP

Saber se P = NP ou P ≠ NP é um problema em aberto desde 1971 e um dos sete Problemas do Milênio do Instituto Clay. A dúvida central é se todo problema cuja resposta pode ser conferida rapidamente também pode ser resolvido do zero em tempo polinomial (Figura 2).

A maioria dos pesquisadores acredita que P ≠ NP porque milhares de problemas práticos já foram provados NP-Completos em áreas bem diferentes e, em mais de 50 anos de

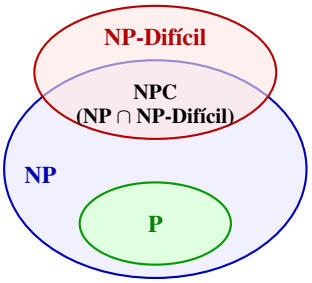


Figura 2. Relação entre P, NP, NPC e NP-Difícil supondo P $\neq$ NP [1].

estudo, nenhum algoritmo exato polinomial foi encontrado para nenhum deles [1]. Além disso, se P = NP com um algoritmo prático, haveria duas consequências diretas:

1) *Criptografia:* algoritmos como RSA e chaves baseadas em logaritmo discreto dependem do fato de que conferir uma chave ou multiplicar primos é rápido (P), enquanto fatorar ou inverter a operação sem a chave é custoso (NP). Se P = NP, essas chaves poderiam ser quebradas em tempo polinomial.
2) *Otimização:* problemas difíceis de logística, escala industrial e acoplamento molecular passariam a ter solução ótima exata em tempo polinomial.

E. Reduções Polinomiais e Demonstrações

Para provar que um novo problema L é NP-Completo sem reduzir toda a classe NP até ele, usa-se o Lema 34.8:

Lema 2 (Lema 34.8 de Cormen et al. [1])**.** *Se L' é NP-Completo e $L' \leq_P L$, então L é NP-Difícil. Se, além disso, $L \in$ NP, então $L \in$ NPC.*

Com isso, o roteiro de prova no Capítulo 34 de Cormen et al. [1] segue cinco passos: (1) provar que $L \in$ NP; (2) escolher um $L' \in$ NPC conhecido; (3) descrever a transformação $f(x)$; (4) provar que $x \in L' \iff f(x) \in L$; e (5) mostrar que f roda em tempo polinomial. A Figura 3 mostra a sequência de reduções usada até chegar em `VERTEX-COVER` e `TSP`.

1) De CIRCUIT-SAT para SAT e 3-CNF-SAT: O primeiro problema NP-Completo da cadeia é a satisfazibilidade de circuitos booleanos (`CIRCUIT-SAT`, Teorema 34.7 de Cormen et al. [1]). Para mostrar que toda linguagem $L \in$ NP se reduz a `CIRCUIT-SAT` (Lema 34.6), considera-se o verificador $A(x, y)$ que roda em tempo $T(n) = O(n^k)$. Em cada etapa da execução, o estado da memória, do contador de programa e dos registradores forma uma configuração c_i. Como a passagem de c_i para c_{i+1} é feita pelo hardware usando portas `AND`, `OR` e `NOT` (circuito M), basta ligar $T(n)$ cópias de M em sequência, fixar a entrada x e deixar os bits do certificado y como entradas livres. O circuito resultante $C = f(x)$ tem tamanho polinomial e produz saída 1 se e somente se existir um certificado y tal que $A(x, y) = 1$.

Para reduzir CIRCUIT-SAT $\leq_P$ SAT (Teorema 34.9), não podemos simplesmente substituir as entradas das portas de trás para frente, pois fios com leque de saída (*fan-out*) ≥ 2 duplicariam subfórmulas e fariam a fórmula crescer de forma exponencial. Em vez disso, cria-se uma variável x_i para cada fio e escreve-se uma cláusula com o operador $\leftrightarrow$ para cada porta (por exemplo, $x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)$ para uma porta `AND` na Figura 34.10 do livro). A fórmula final é o `AND` do fio de saída com as cláusulas de todas as portas.

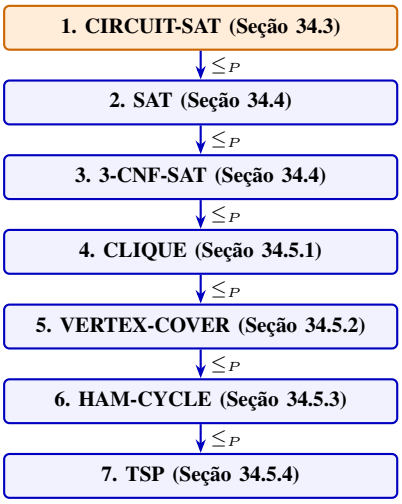


Figura 3. Sequência de reduções polinomiais do Capítulo 34 de Cormen et al. [1].

Depois, reduz-se SAT $\leq_P$ 3-CNF-SAT (Teorema 34.10), onde cada cláusula é um `OR` de exatamente três literais distintos. A conversão é feita em três etapas: (i) monta-se a árvore binária da fórmula criando variáveis y_i nos nós internos para que cada cláusula tenha no máximo 3 variáveis; (ii) usa-se a tabela-verdade de cada cláusula (máximo de 8 linhas) e as Leis de De Morgan para passá-la para a Forma Normal Conjuntiva (CNF); e (iii) usam-se duas variáveis auxiliares p e q para esticar cláusulas de 1 ou 2 literais (por exemplo, $(l_1 \vee l_2)$ vira $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$).

2) Redução 3-CNF-SAT $\leq_P$ CLIQUE: Um clique em $G = (V, E)$ é um subconjunto $V' \subseteq V$ onde todo par de vértices é ligado por uma aresta. O problema `CLIQUE` pergunta se existe um clique de tamanho k em G.

Teorema 2 (Teorema 34.11 de Cormen et al. [1])**.** *O problema CLIQUE é NP-Completo.*

Demonstração. Primeiro, CLIQUE $\in$ NP: dado o conjunto V' como certificado, confere-se se $|V'| = k$ e se toda dupla $u, v \in V'$ tem aresta em E em tempo $O(|V|^2)$.

Para reduzir 3-CNF-SAT $\leq_P$ CLIQUE, seja $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ uma fórmula em 3-CNF em que cada cláusula é $C_r = (l_1^r \vee l_2^r \vee l_3^r)$. Construímos o grafo $G = (V, E)$ assim:

- Para cada cláusula C_r, colocamos uma tripla de vértices $V_r = \{v_1^r, v_2^r, v_3^r\}$ em V ($3k$ vértices ao todo).
- Colocamos uma aresta entre v_i^r e v_j^s se: (a) eles estão em triplas diferentes ($r \neq s$); e (b) seus literais são coerentes, isto é, l_i^r não é a negação de l_j^s ($l_i^r \neq \neg l_j^s$).

Essa construção testa os pares entre os $3k$ vértices e roda em tempo polinomial $O(k^2)$. Resta provar a equivalência:

- *Ida ($\implies$):* Se ϕ tem uma atribuição que a satisfaz, cada cláusula C_r tem pelo menos um literal verdadeiro $l_i^r = 1$. Escolhendo um vértice verdadeiro v_i^r de cada tripla, formamos V' com k vértices. Como dois vértices de V' têm literais iguais a 1 na mesma atribuição, um não pode ser a negação do outro. Logo, todos os pares em V' têm aresta em E, formando um clique de tamanho k.
- *Volta ($\impliedby$):* Se G tem um clique V' de tamanho k, como não há arestas dentro da mesma tripla V_r, V' tem exatamente um vértice de cada uma das k triplas. Como os vértices de V' são todos ligados entre si, nenhum literal de V' é a negação de outro, permitindo atribuir 1 a todos os literais de V' sem conflito e satisfazer ϕ.

□

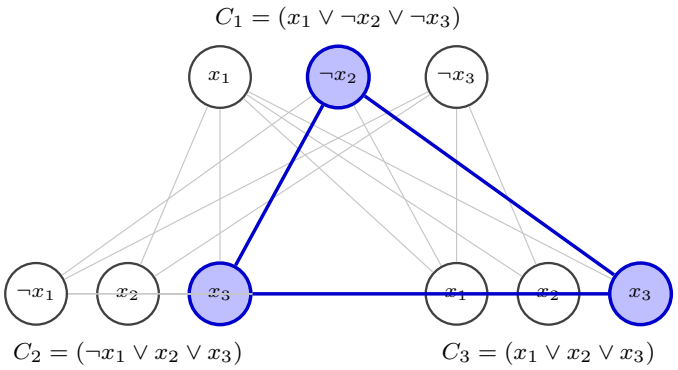


Figura 4. Redução 3-CNF-SAT $\leq_P$ CLIQUE da Figura 34.14 de Cormen et al. [1]. Para $x_2 = 0$ e $x_3 = 1$, os vértices $\{\neg x_2, x_3, x_3\}$ formam um clique de tamanho 3.

3) Redução CLIQUE $\leq_P$ VERTEX-COVER e TSP: Dado $G = (V, E)$, o grafo complemento é $\overline{G} = (V, \overline{E})$, com $\overline{E} = \{(u, v) : u \neq v \text{ e } (u, v) \notin E\}$ (Figura 5).

Teorema 3 (Teorema 34.12 de Cormen et al. [1])**.** *O problema VERTEX-COVER é NP-Completo.*

Demonstração. Já vimos que VERTEX-COVER $\in$ NP. Dada uma instância (G, k) de `CLIQUE`, calcula-se o complemento $\overline{G}$ em $O(|V|^2)$ e gera-se a instância $(\overline{G}, |V| - k)$ de `VERTEX-COVER`:

- *Ida ($\implies$):* Se V' é um clique de tamanho k em G, então para toda aresta $(u, v) \in \overline{E}$ vale $(u, v) \notin E$. Logo, u e v não podem estar ambos em V', ou seja, pelo menos um deles pertence a $V \setminus V'$. Assim, $V \setminus V'$ cobre todas as arestas de $\overline{E}$ e tem tamanho $|V| - k$.
- *Volta ($\impliedby$):* Se V' é uma cobertura de tamanho $|V| - k$ em $\overline{G}$, então para toda dupla $u, v \in V \setminus V'$ a aresta (u, v) não pode estar em $\overline{E}$ (pois não teria ponta em V'). Logo, $(u, v) \in E$ e $V \setminus V'$ é um clique de tamanho k em G.

□

Em seguida, o Teorema 34.13 reduz VERTEX-COVER $\leq_P$ HAM-CYCLE trocando cada aresta $(u, v) \in E$ por um *widget*

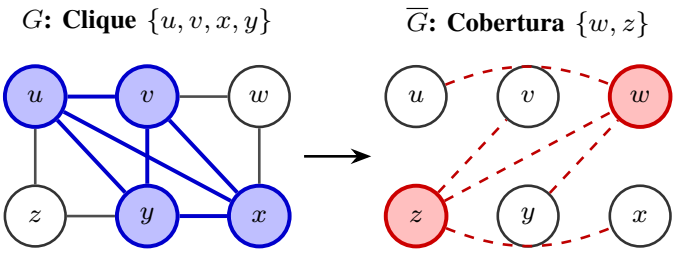


Figura 5. Redução CLIQUE $\leq_P$ VERTEX-COVER (Figura 34.15 de Cormen et al. [1]).

W_{uv} de 12 vértices ligado a k vértices seletores $s_1, \ldots, s_k$. Por fim, o Teorema 34.14 prova que HAM-CYCLE $\leq_P$ TSP: dado $G = (V, E)$, monta-se o grafo completo G' com custo $c(i, j) = 0$ se $(i, j) \in E$ e $c(i, j) = 1$ se $(i, j) \notin E$, e limite $k = 0$. Assim, G tem ciclo hamiltoniano se e somente se G' tem percurso de custo 0.

F. Explosão Combinatória

A dificuldade prática dos algoritmos de força bruta vem da quantidade de combinações possíveis:

1) *Cobertura de Vértices (2^n):* Como cada um dos $n = |V|$ vértices pode entrar ou não no subconjunto, o total de subconjuntos é $\sum_{k=0}^{n} \binom{n}{k} = 2^n$, levando a $O(2^n \cdot |E|)$.
2) *Caixeiro-Viajante ($(n-1)!/2$):* Fixando a primeira cidade entre as $n!$ permutações ($n!/n = (n-1)!$) e dividindo por 2 porque o sentido horário e anti-horário têm o mesmo custo em grafos simétricos, existem $\frac{(n-1)!}{2}$ rotas.

Aumentar a velocidade do processador ajuda pouco quando o algoritmo é exponencial. Se um algoritmo de tempo 2^n passa a rodar em um computador 1.000 vezes mais rápido, o novo tamanho resolvido no mesmo tempo é $2^{n'} = 1000 \cdot 2^n \implies n' = n + \log_2(1000) \approx n + 10$. Ou seja, ganham-se apenas 10 vértices. A Tabela I mostra o tempo estimado em um processador de 10^9 operações por segundo. Quando n cresce, usam-se algoritmos de aproximação com razão $\max(C/C^*, C^*/C) \leq \rho(n)$ (Cap. 35 [1]) ou metaheurísticas como o *Simulated Annealing*.

Tabela I
TEMPO ESTIMADO PARA 10^9 OP/S EM DIFERENTES COMPLEXIDADES.

n	Polinomial (n^2)	Exponencial (2^n)	Held-Karp ($n^2 2^n$)	Fatorial ($(n-1)!/2$)
5	25 ns	32 ns	0,8 μs	12 ns
10	100 ns	1,0 μs	102,4 μs	181,4 μs
15	225 ns	32,8 μs	7,4 ms	43,6 s
20	400 ns	1,0 ms	419,4 ms	1,9 anos
25	625 ns	33,6 ms	21,0 s	$9{,}8 \times 10^6$ anos
50	2,5 μs	13,0 dias	89,3 anos	$9{,}6 \times 10^{45}$ anos
100	10,0 μs	$4{,}0 \times 10^{13}$ anos	$4{,}0 \times 10^{17}$ anos	$1{,}5 \times 10^{139}$ anos

Algoritmo 1: APPROX-VERTEX-COVER(G) [1]

Entrada: Grafo não dirigido $G = (V, E)$.
Saída: Cobertura de vértices $C \subseteq V$ com $|C| \leq 2|C^*|$.

$C \leftarrow \emptyset$
$E' \leftarrow E[G]$
while $E' \neq \emptyset$ **do**
    Escolha uma aresta qualquer $(u, v) \in E'$
    $C \leftarrow C \cup \{u, v\}$
    Remova de E' todas as arestas ligadas a u ou v
return C

Tabela II
RESUMO DE ONDE CADA ABORDAGEM FUNCIONA E ONDE FALHA.

Abordagem	Onde Usar	Onde Falha
Exatos (Força Bruta, B&B, Held-Karp)	Entradas pequenas ($n \leq 20$) e testes que exigem o ótimo exato C^*.	Explosão de tempo ($2^n, n!$) e falta de memória RAM ($\Theta(n2^n)$).
Aproximação (Cap. 35)	Grafos grandes quando há propriedade estrutural (ex: desigualdade triangular).	Problemas sem aproximação constante (TSP geral) ou domínios contínuos.
Metaheurísticas (*Simulated Annealing*)	Espaços contínuos ou grandes com vários mínimos locais (*Docking*).	Quando se exige garantia formal de erro ou o orçamento B é muito curto.

III. O Paradigma Algorítmico (Engenharia da Solução)

IV. Análise Assintótica Rigorosa (Custos)

V. Estudo de Caso e Implementação Prática

VI. Conclusões

Este trabalho uniu o estudo teórico dos Capítulos 34 e 35 de Cormen et al. [1] com testes práticos de algoritmos exatos, aproximados e metaheurísticos em Python. A partir dos experimentos da Seção V, foi possível observar na prática quando cada abordagem funciona bem e quando deixa de ser viável.

Nos algoritmos exatos e verificadores, ficou evidente a diferença entre verificar e resolver um problema. Enquanto os verificadores validam uma solução de TSP em $O(n)$ e de *Vertex Cover* em $O(|V| + |E|)$ em poucos microssegundos, a força bruta trava logo em entradas pequenas. No TSP com $n = 10$, a Força Bruta levou 1,522 s, enquanto o algoritmo de Held-Karp [7] ($\Theta(n^2 2^n)$) levou 6,92 ms (220 vezes mais rápido) e resolveu $n = 14$ em 235,1 ms. Na Cobertura de Vértices com $n = 18$, o *Branch-and-Bound* caiu de 242,1 ms (força bruta) para 4,97 ms. Mesmo assim, métodos exatos só servem para entradas pequenas: acima de $n = 22$, a tabela $\Theta(n2^n)$ do Held-Karp estoura a memória RAM.

Já os algoritmos de aproximação rodaram em microssegundos e mantiveram o erro dentro dos limites teóricos do Capítulo 35: no APPROX-VERTEX-COVER ($\rho = 2$), a razão média ficou entre 1,42 e 1,70; no APPROX-TSP-TOUR ($\rho = 2$ com desigualdade triangular), as rotas ficaram em média de 8% a 20% acima do ótimo; e no GREEDY-SET-COVER ($\rho = H(d)$), a razão média ficou entre 1,00 e 1,06.

Por fim, no problema contínuo inspirado em *Molecular Docking* [2], [3], o *Simulated Annealing* dependeu diretamente do orçamento de avaliações B. Com poucas avaliações ($B = 100$), a temperatura ainda alta faz o algoritmo aceitar pioras e perder para o *Random Search*. Já para $B \geq 10.000$, o resfriamento gradual permitiu sair de mínimos locais e superar a busca aleatória em até 29 das 30 execuções ($p = 5{,}77 \times 10^{-8}$). A Tabela II resume onde cada método se aplica.

Referências

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Algoritmos: Teoria e Prática*, 3rd ed. Rio de Janeiro, Brasil: Elsevier, 2012.

[2] N. S. Pagadala, K. Syed, and J. Tuszynski, "Software for molecular docking: A review," *Biophys. Rev.*, vol. 9, no. 2, pp. 91–102, 2017.

[3] S.-Y. Huang, "Comprehensive assessment of flexible-ligand docking algorithms: Current effectiveness and challenges," *Briefings Bioinf.*, vol. 19, no. 5, pp. 982–994, 2018.

[4] S. A. Cook, "The complexity of theorem-proving procedures," in *Proc. 3rd Annu. ACM Symp. Theory of Computing (STOC)*, 1971, pp. 151–158.

[5] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*. New York, NY, USA: Plenum Press, 1972, pp. 85–103.

[6] L. A. Levin, "Universal sequential search problems," *Problemy Peredachi Informatsii*, vol. 9, no. 3, pp. 115–116, 1973.

[7] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *J. Soc. Ind. Appl. Math.*, vol. 10, no. 1, pp. 196–210, 1962.

[8] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[9] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *J. Chem. Phys.*, vol. 21, no. 6, pp. 1087–1092, 1953.

Apêndice A
Cheat Sheet: Guia de Consulta Rápida

a) Definição em Uma Frase

Quando um problema tem verificação rápida em tempo polinomial (NP), mas achar o ótimo exato sofre explosão combinatória (NP-Completo), usam-se métodos exatos com poda apenas para entradas pequenas e recorre-se a aproximações (com limite $\rho(n)$) ou metaheurísticas para entradas maiores.

b) Quando Utilizar Cada Abordagem?

c) Tabela Resumo de Complexidades

Algoritmo	Tempo (Pior Caso)	Espaço	Garantia
Verificador Vertex Cover	$O(\|V\| + \|E\|)$	$O(\|V\|)$	Exata (Decisão)
Verificador TSP	$O(n)$	$O(n)$	Exata (Decisão)
Vertex Cover (Força Bruta)	$O(2^n \cdot \|E\|)$	$O(n)$	Ótimo (C^*)
Vertex Cover (Branch-and-Bound)	$O(2^k \cdot \|E\|)$	$O(n)$	Ótimo (C^*)
TSP (Força Bruta)	$\Theta((n-1)! \cdot n)$	$O(n)$	Ótimo (C^*)
TSP (Held-Karp / PD)	$\Theta(n^2 2^n)$	$\Theta(n2^n)$	Ótimo (C^*)
APPROX-VERTEX-COVER	$O(\|V\| + \|E\|)$	$O(\|V\|)$	$\rho = 2$
APPROX-TSP-TOUR	$O(n^2)$	$O(n)$	$\rho = 2$ (métrico)
GREEDY-SET-COVER	$O(\|X\|\|\mathcal{F}\| \min(\|X\|, \|\mathcal{F}\|))$	$O(\|X\|)$	$\rho = H(d)$
Simulated Annealing	$O(A \cdot C)$	$O(1)$	Empírica

d) Esqueleto Base da Técnica